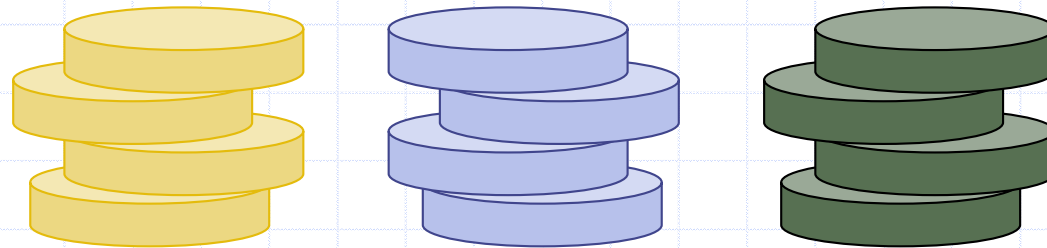


Presentation for use with the textbook **Data Structures and Algorithms in Java, 6th edition**, by M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Wiley, 2014

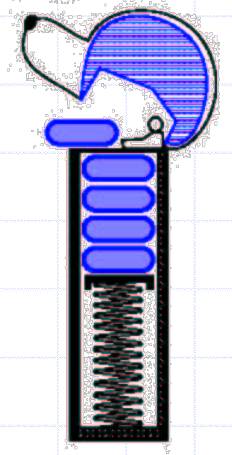
Stacks



Abstract Data Types (ADTs)

- An abstract data type (ADT) is an abstraction of a data structure
- An ADT specifies:
 - Data stored
 - Operations on the data
 - Error conditions associated with operations
- Example: ADT modeling a simple stock trading system
 - The data stored are buy/sell orders
 - The operations supported are
 - ♦ order **buy**(stock, shares, price)
 - ♦ order **sell**(stock, shares, price)
 - ♦ void **cancel**(order)
 - Error conditions:
 - ♦ Buy/sell a nonexistent stock
 - ♦ Cancel a nonexistent order

The Stack ADT



- ❑ The **Stack** ADT stores arbitrary objects
- ❑ Insertions and deletions follow the last-in first-out scheme
- ❑ Think of a spring-loaded plate dispenser
- ❑ Main stack operations:
 - **push**(object): inserts an element
 - object **pop**(): removes and returns the last inserted element
- ❑ Auxiliary stack operations:
 - object **top**(): returns the last inserted element without removing it
 - integer **size**(): returns the number of elements stored
 - boolean **isEmpty**(): indicates whether no elements are stored

Stack Interface in Java

- ❑ Java interface corresponding to our Stack ADT
- ❑ Assumes null is returned from `top()` and `pop()` when stack is empty
- ❑ Different from the built-in Java class `java.util.Stack`

```
public interface Stack<E> {  
    int size();  
    boolean isEmpty();  
    E top();  
    void push(E element);  
    E pop();  
}
```

Example

Method	Return Value	Stack Contents
push(5)	—	(5)
push(3)	—	(5, 3)
size()	2	(5, 3)
pop()	3	(5)
isEmpty()	false	(5)
pop()	5	()
isEmpty()	true	()
pop()	null	()
push(7)	—	(7)
push(9)	—	(7, 9)
top()	9	(7, 9)
push(4)	—	(7, 9, 4)
size()	3	(7, 9, 4)
pop()	4	(7, 9)
push(6)	—	(7, 9, 6)
push(8)	—	(7, 9, 6, 8)
pop()	8	(7, 9, 6)

Exceptions vs. Returning Null

- ❑ Attempting the execution of an operation of an ADT may sometimes cause an error condition
- ❑ Java supports a general abstraction for errors, called exception
- ❑ An exception is said to be “thrown” by an operation that cannot be properly executed
- ❑ In our Stack ADT, we do not use exceptions
- ❑ Instead, we allow operations pop and top to be performed even if the stack is empty
- ❑ For an empty stack, pop and top simply return null

Applications of Stacks

- Direct applications
 - Page-visited history in a Web browser
 - Undo sequence in a text editor
 - Chain of method calls in the Java Virtual Machine
- Indirect applications
 - Auxiliary data structure for algorithms
 - Component of other data structures

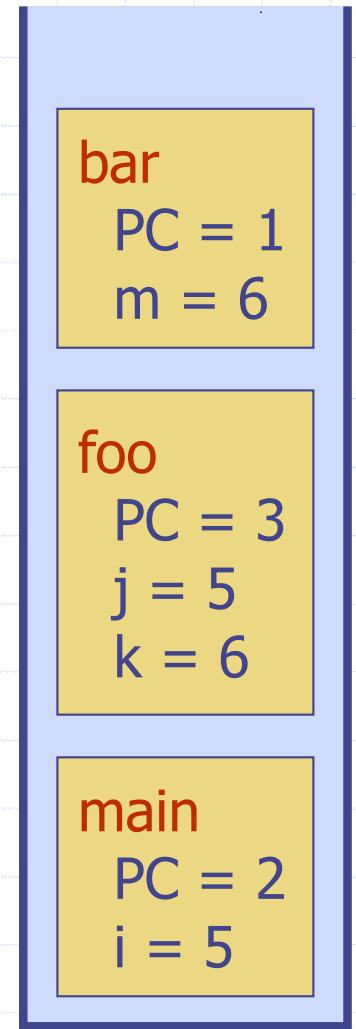
Method Stack in the JVM

- ❑ The Java Virtual Machine (JVM) keeps track of the chain of active methods with a stack
- ❑ When a method is called, the JVM pushes on the stack a frame containing
 - Local variables and return value
 - Program counter, keeping track of the statement being executed
- ❑ When a method ends, its frame is popped from the stack and control is passed to the method on top of the stack
- ❑ Allows for **recursion**

```
main() {  
    int i = 5;  
    foo(i);  
}
```

```
foo(int j) {  
    int k;  
    k = j+1;  
    bar(k);  
}
```

```
bar(int m) {  
    ...  
}
```

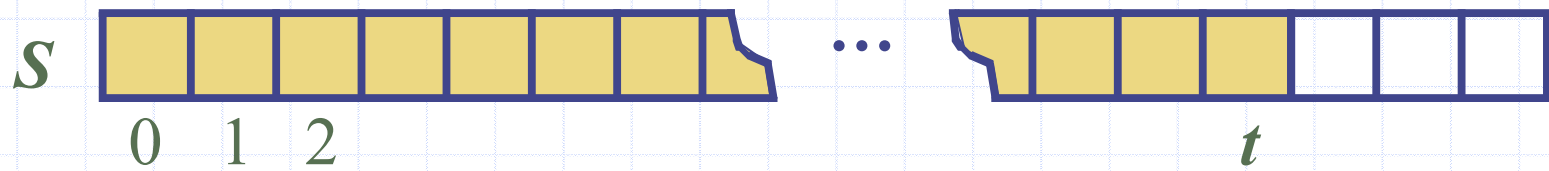


Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element

Algorithm *size()*
return $t + 1$

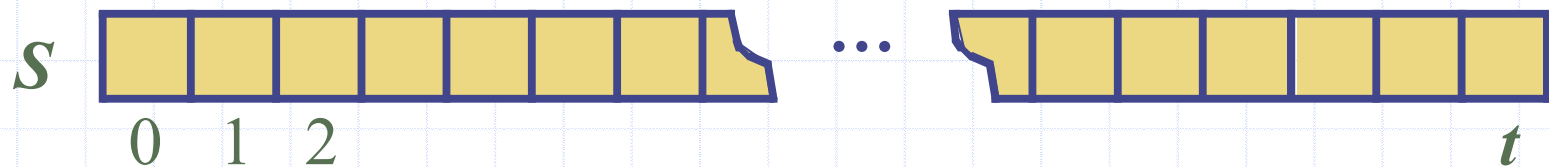
Algorithm *pop()*
if *isEmpty()* then
 return null
else
 $t \leftarrow t - 1$
 return $S[t + 1]$



Array-based Stack (cont.)

- The array storing the stack elements may become full
- A push operation will then throw a **FullStackException**
 - Limitation of the array-based implementation
 - Not intrinsic to the Stack ADT

```
Algorithm push(o)  
  if  $t = S.length - 1$  then  
    throw IllegalStateException  
  else  
     $t \leftarrow t + 1$   
     $S[t] \leftarrow o$ 
```



Performance and Limitations

□ Performance

- Let n be the number of elements in the stack
- The space used is $O(n)$
- Each operation runs in time $O(1)$

□ Limitations

- The maximum size of the stack must be defined a priori and cannot be changed
- Trying to push a new element into a full stack causes an implementation-specific exception

Array-based Stack in Java

```
public class ArrayStack<E>
    implements Stack<E> {

    // holds the stack elements
    private E[] S;

    // index to top element
    private int top = -1;

    // constructor
    public ArrayStack(int capacity) {
        S = (E[]) new Object[capacity];
    }
}
```

```
public E pop() {
    if isEmpty()
        return null;
    E temp = S[top];
    // facilitate garbage collection:
    S[top] = null;
    top = top - 1;
    return temp;
}

... (other methods of Stack interface)
```

Example Use in Java

```
public class Tester {  
    // ... other methods  
    public intReverse(Integer a[]) {  
        Stack<Integer> s;  
        s = new ArrayStack<Integer>();  
        ... (code to reverse array a) ...  
    }  
}
```

```
public floatReverse(Float f[]) {  
    Stack<Float> s;  
    s = new ArrayStack<Float>();  
    ... (code to reverse array f) ...  
}
```

Parentheses Matching

- Each “(”, “{”, or “[” must be paired with a matching “)”, “}”, or “]”
 - correct: ()(()){([())}
 - correct: ((())(()){([())}
 - incorrect:)(()){([())}
 - incorrect: ({ []})
 - incorrect: (

Parenthesis Matching (Java)

```
public static boolean isMatched(String expression) {  
    final String opening = "{["; // opening delimiters  
    final String closing = "}]"; // respective closing delimiters  
    Stack<Character> buffer = new LinkedStack<>( );  
    for (char c : expression.toCharArray( )) {  
        if (opening.indexOf(c) != -1) // this is a left delimiter  
            buffer.push(c);  
        else if (closing.indexOf(c) != -1) { // this is a right delimiter  
            if (buffer.isEmpty( )) // nothing to match with  
                return false;  
            if (closing.indexOf(c) != opening.indexOf(buffer.pop( )))  
                return false; // mismatched delimiter  
        }  
    }  
    return buffer.isEmpty( ); // were all opening delimiters matched?  
}
```

HTML Tag Matching

- ❑ For fully-correct HTML, each `<name>` should pair with a matching `</name>`

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

HTML Tag Matching (Java)

```
public static boolean isHTMLMatched(String html) {  
    Stack<String> buffer = new LinkedList<>( );  
    int j = html.indexOf('<'); // find first '<' character (if any)  
    while (j != -1) {  
        int k = html.indexOf('>', j+1); // find next '>' character  
        if (k == -1)  
            return false; // invalid tag  
        String tag = html.substring(j+1, k); // strip away < >  
        if (!tag.startsWith("/")) // this is an opening tag  
            buffer.push(tag);  
        else { // this is a closing tag  
            if (buffer.isEmpty( ))  
                return false; // no tag to match  
            if (!tag.substring(1).equals(buffer.pop( )))  
                return false; // mismatched tag  
        }  
        j = html.indexOf('<', k+1); // find next '<' character (if any)  
    }  
    return buffer.isEmpty( ); // were all opening tags matched?  
}
```

Evaluating Arithmetic Expressions

Slide by Matt Stallmann
included with permission.

$$14 - 3 * 2 + 7 = (14 - (3 * 2)) + 7$$

Operator precedence

* has precedence over +/−

Associativity

operators of the same precedence group
evaluated from left to right

Example: $(x - y) + z$ rather than $x - (y + z)$

Idea: push each operator on the stack, but first pop and perform higher and *equal* precedence operations.

Algorithm for Evaluating Expressions

Slide by Matt Stallmann
included with permission.

Two stacks:

- opStk holds operators
- valStk holds values
- Use \$ as special “end of input” token with lowest precedence

Algorithm **doOp()**

```
x ← valStk.pop();  
y ← valStk.pop();  
op ← opStk.pop();  
valStk.push( y op x )
```

Algorithm **repeatOps(refOp)**:

```
while ( valStk.size() > 1 ∧  
        prec(refOp) ≤  
        prec(opStk.top())  
doOp()
```

Algorithm **EvalExp()**

Input: a stream of tokens representing
an arithmetic expression (with
numbers)

Output: the value of the expression

while there's another token z

if isNumber(z) **then**

valStk.push(z)

else

repeatOps(z);

opStk.push(z)

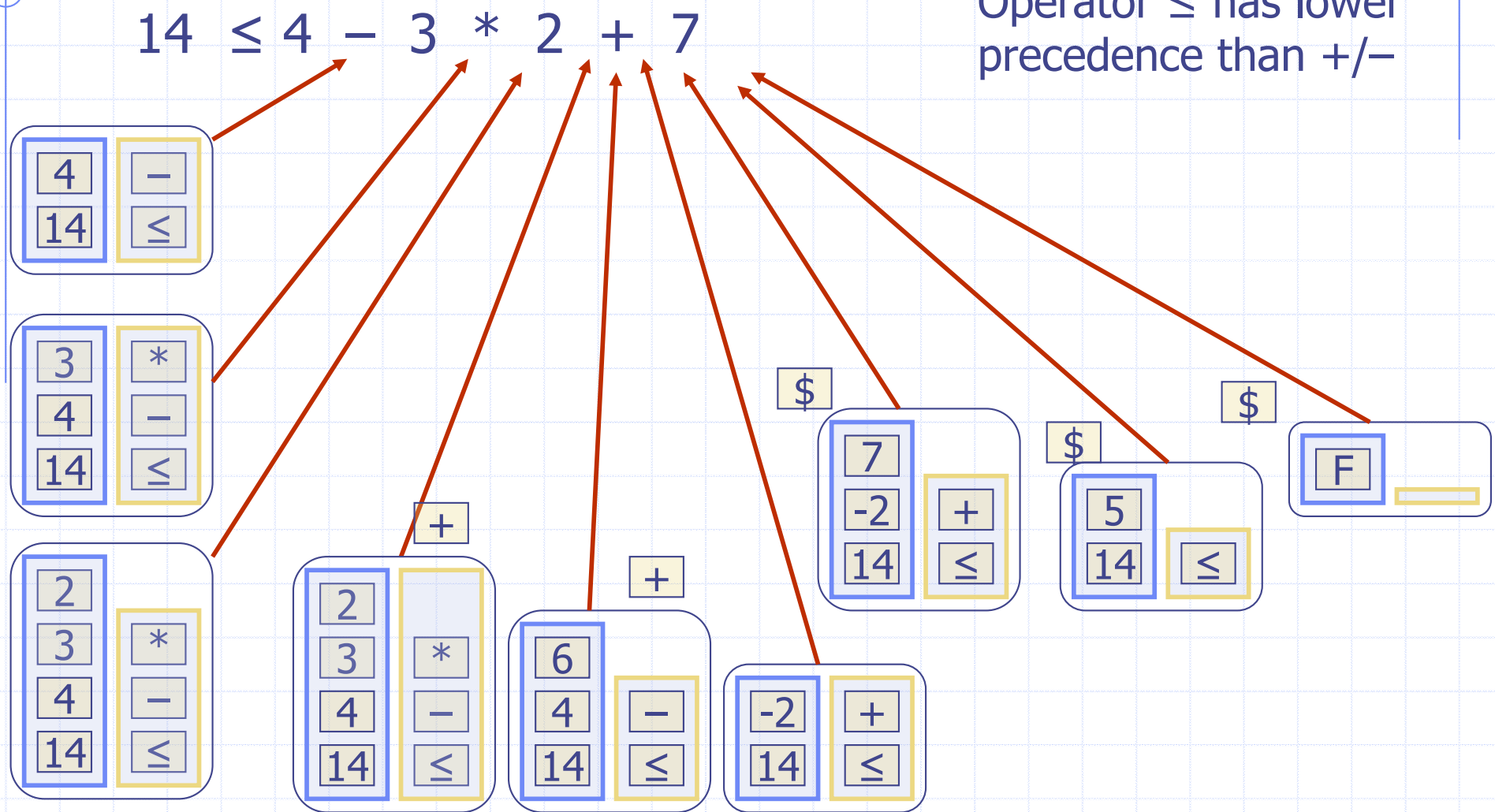
repeatOps(\$);

return valStk.top()

Algorithm on an Example Expression

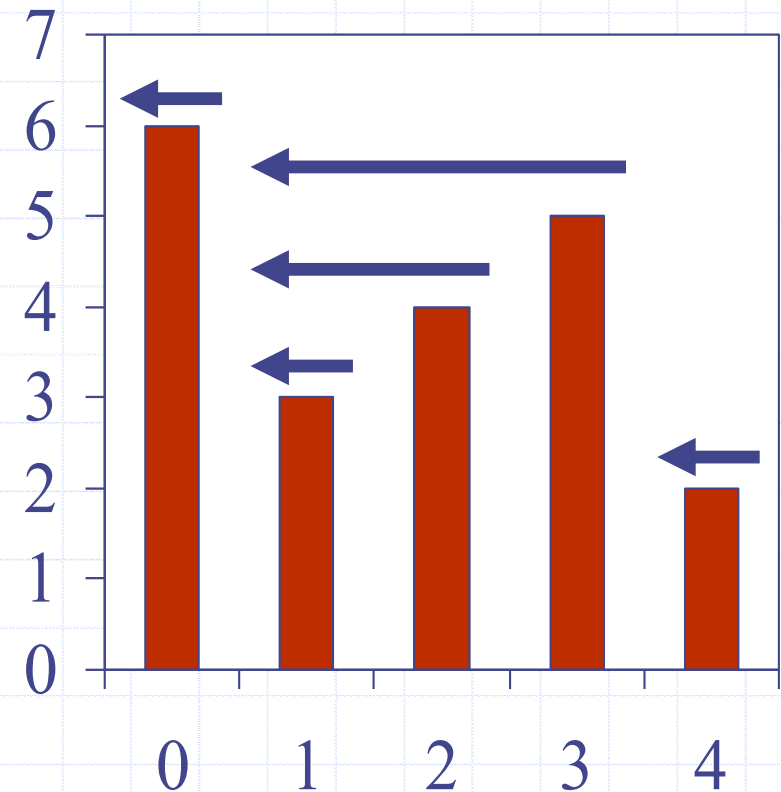
Slide by Matt Stallmann
included with permission.

Operator $\leq$ has lower
precedence than $+/-$



Computing Spans (not in book)

- Using a stack as an auxiliary data structure in an algorithm
- Given an array X , the **span** $S[i]$ of $X[i]$ is the maximum number of consecutive elements $X[j]$ immediately preceding $X[i]$ and such that $X[j] \leq X[i]$
- Spans have applications to financial analysis
 - E.g., stock at 52-week high



X	6	3	4	5	2
S	1	1	2	3	1

Quadratic Algorithm

Algorithm *spans1*(X, n)

Input array X of n integers

Output array S of spans of X

$S \leftarrow$ new array of n integers

for $i \leftarrow 0$ **to** $n - 1$ **do**

$s \leftarrow 1$

while $s \leq i \wedge X[i - s] \leq X[i]$

$s \leftarrow s + 1$

$S[i] \leftarrow s$

return S

#

n

n

n

$1 + 2 + \dots + (n - 1)$

$1 + 2 + \dots + (n - 1)$

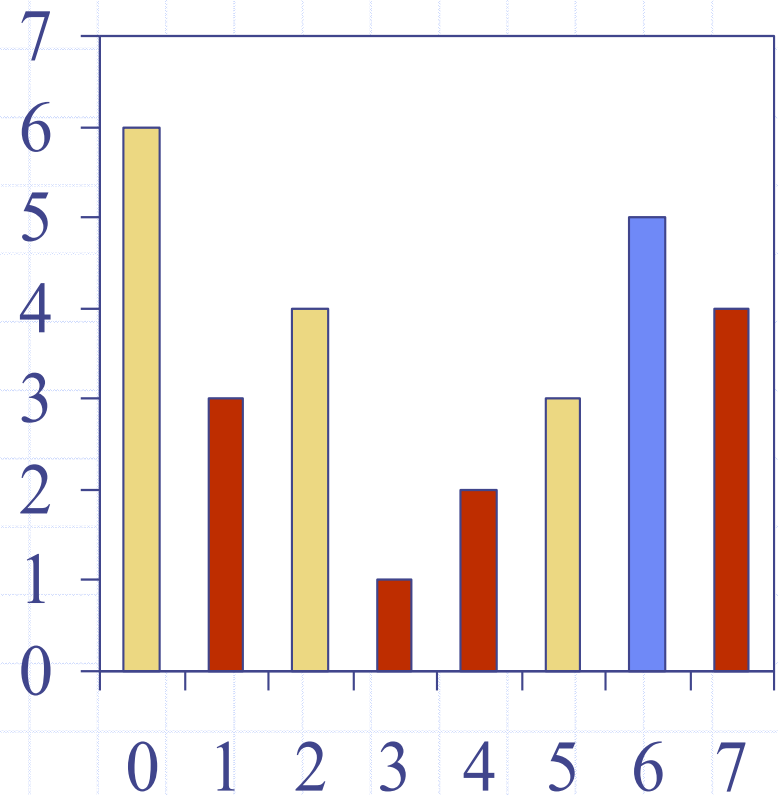
n

1

◆ Algorithm *spans1* runs in $O(n^2)$ time

Computing Spans with a Stack

- We keep in a stack the indices of the elements visible when “looking back”
- We scan the array from left to right
 - Let i be the current index
 - We pop indices from the stack until we find index j such that $X[i] < X[j]$
 - We set $S[i] \leftarrow i - j$
 - We push i onto the stack



Linear Time Algorithm

- Each index of the array
 - Is pushed into the stack exactly one
 - Is popped from the stack at most once
- The statements in the while-loop are executed at most n times
- Algorithm *spans2* runs in $O(n)$ time

Algorithm <i>spans2</i> (X, n)	#
$S \leftarrow$ new array of n integers	n
$A \leftarrow$ new empty stack	1
for $i \leftarrow 0$ to $n - 1$ do	n
while $(\neg A.isEmpty() \wedge$	
$X[A.top()] \leq X[i])$ do	n
$A.pop()$	n
if $A.isEmpty()$ then	n
$S[i] \leftarrow i + 1$	n
else	
$S[i] \leftarrow i - A.top()$	n
$A.push(i)$	n
return S	1